

DG3NLP REPORT

Oliver Blatchford



NUMBER OF WORDS: 1327

Contents

1.Introduction and Background Research 2

2.Requirements 2

3. Design and Implementation 3

4. Evaluation and Conclusion 6

5. Innovation 8

6. References 10

7. Annex 10

1. Introduction and Background Research

The NLP task chosen for this project was emotion classification. This is important because "Emotions are the key to people's feelings and thoughts" (Jabreel and Moreno, 2019). By understanding emotions they can act more naturally and understand the nuances of speech such as sarcasm. When researching other projects multiple different methods were shown to be used. Some used transformers such as BERT as they are "designed to pre-train deep bidirectional representations" (Devlin *et al.*, 2019) by using surrounding context. Many projects used multi-class classification however others argue that multi-label was better and produced more accurate results (Jabreel and Moreno, 2019). Differing studies used deep learning models that automatically "extract features from short texts based on… values" for a "deep convolutional neural network" (Poria, Cambria and Gelbukh, 2015). Other approaches have shown that simple classifiers can still be effective such as (Joulin *et al.*, 2017) showed that linear models trained on embeddings can achieve competitive performance with a smaller computational power requirement. This is why it was decided to use DistilBERT and a multiclass logistic regression to give understanding while remaining efficient.

2. Requirements

The goal of the project was to create a model in python that could classify emotions from tweets as accurately as possible. The training dataset provided by the coursework was used as the data source which contains tweets under the text column and labelled emotions under the label column. To evaluate the models and compare them the scikit-learn library was used and allowed for the creation of classification reports. This created identically structured reports between both models allowing for easy comparison of the metrics. The key metrics are precision, recall, f1-score and accuracy.

Classification Report (for the 6 emotions):

	precision	recall	f1-score	support
Anger (0)	0.92	0.93	0.92	933
Fear (1)	0.88	0.93	0.91	1072
Joy (2)	0.82	0.74	0.78	261
Love (3)	0.90	0.87	0.88	432
Sadness (4)	0.84	0.83	0.84	387
Surprise (5)	0.82	0.73	0.77	115
accuracy			0.89	3200
macro avg	0.87	0.84	0.85	3200
weighted avg	0.89	0.89	0.89	3200

Figure 1 - Classification Report for Logistic Regression

Classification Report (for the 6 emotions):

	precision	recall	f1-score	support
Anger (0)	0.97	0.96	0.97	933
Fear (1)	0.97	0.92	0.94	1072
Joy (2)	0.75	0.95	0.84	261
Love (3)	0.95	0.92	0.94	432
Sadness (4)	0.87	0.97	0.92	387
Surprise (5)	0.94	0.68	0.79	115
accuracy			0.93	3200
macro avg	0.91	0.90	0.90	3200
weighted avg	0.94	0.93	0.93	3200

Saved in './distilbert_emotion_model' directory.

Figure 2 - Classification report for DistilBERT

The data given did not require preprocessing as it only contained text. However, preprocessing was still implemented to ensure that the emotion detector works with other datasets. This is because tweets are noisy data and by removing unnecessary parts like URLs it reduces the vocabulary size which helps the models focus on meaningful words that would help predict the emotion.

As seen on figure 3 an isinstance() method was used to type check that each tweet was text and if wasn't it was converted to an empty string to not cause an error due to the string function. A text.lower() function was used to lowercase the text similarly. There were also series of regular

expressions to remove potential irrelevant additions to tweets such as links and punctuation. The hashtags were removed but the word after it was retained as it could be important to the emotion sentiment of the tweet.

```
#Preprocessing
def clean_text(text):
    if not isinstance(text, str): #if not a string replace with empty string
        text = ''

    text = text.lower()
    text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE) # Remove links
    text = re.sub(r'#', '', text) #Remove hashtags but keeps word after it
    text = re.sub(r'[^\w\s]', '', text) # Removes punctuation
    return text
```

Figure 3 -Preprocessing

3. Design and Implementation

The data pipeline had three main steps, preprocessing, training and evaluation.

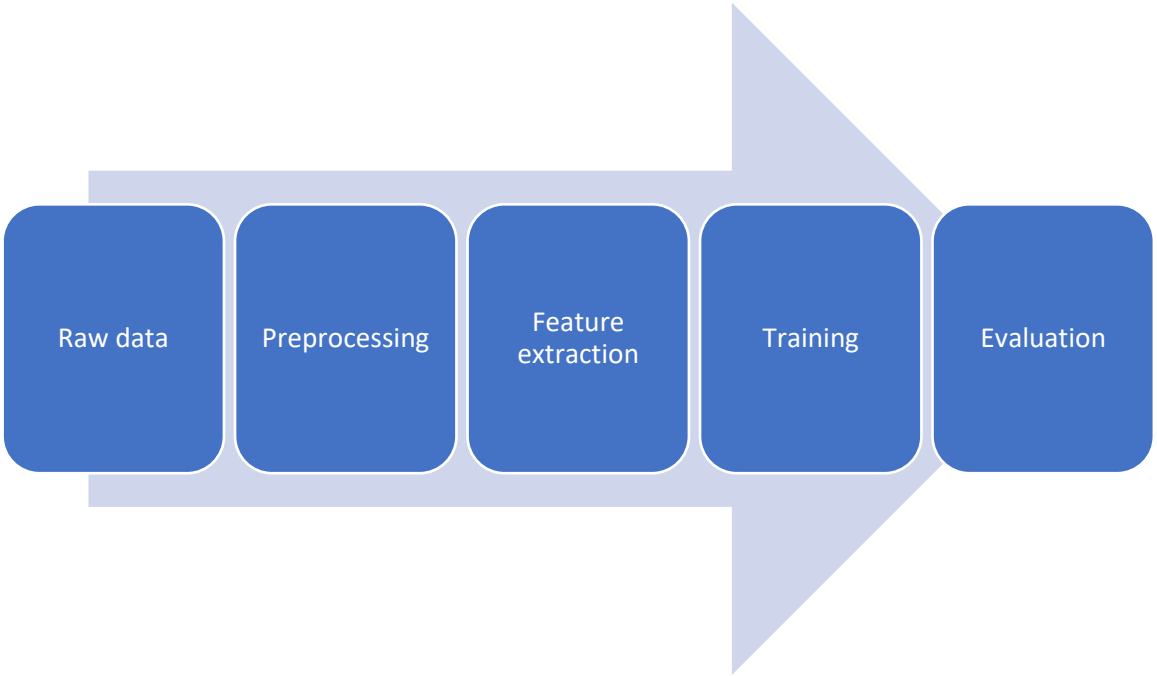


Figure 4 – Continuous block diagram of the data pipeline

The first stage was preprocessing which involved cleaning a huge amount of raw data and removing unnecessary details or parts of the tweets that could create errors such as links or punctuation.

```python
#Preprocessing
def clean_text(text):
    if not isinstance(text, str): #if not a string replace with empty string
        text = ''

    text = text.lower()
    text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE) # Remove links
    text = re.sub(r'#', '', text) #Remove hashtags but keeps word after it
    text = re.sub(r'[^\w\s]', '', text) # Removes punctuation
    return text
```

Figure 5 - Preprocessing (as shown in section 2)

The data also needed to be transformed into a format understandable by the machine, so CountVectorizer was used for logistic regression and AutoTokenizer for fine tuning a transformer.

```python
# change the text into numbers
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(texts.astype('U'))
```

Figure 6 - Count Vectorizer for Logistic Regression

```python
# the models translator so it understands inputs
tokenizer = AutoTokenizer.from_pretrained(model_name)
print("Training and testing data split and model translator loaded")


# Converts the text to be understandable by model. Truncates if too long. Adds padding if too short.
train_encodings = tokenizer(
    list(x_train_text),
    truncation=True,
    padding=True
)
test_encodings = tokenizer(
    list(x_test_text),
    truncation=True,
    padding=True
)
```

Figure 7 – AutoTokenizer for DistilBERT

At this stage the training (80%) and testing (20%) data was split up. The training data was used to fine tune the two models to better predict what emotion each tweet contained. The testing data would later be used to compare the performance of the two models to compare the difference of the simpler and more complex models. To ensure that there was an equal distribution of each emotion in both sets stratify was used so if 10% of the data was anger it would be 10% in both the training and testing.

```python
# split training data 80% and testing data 20%
X_train, X_test, y_train, y_test = train_test_split(X, labels, test_size=0.2, random_state=42,stratify=labels)
```

Figure 8 – Splitting training and testing data

To get a fair evaluation the same evaluation metric of the classification report from sci-kit was used which details key model metrics such as precision, recall and F1-score into a easily readable table.

```
#Creates report showing precision, recall, and f1-score for the emotions
print("Classification Report (for the 6 emotions):")
print(classification_report(y_test, y_pred,target_names=emotion_labels))
print("\n")
```

Figure 9 – Creating classification report for both models

In this project it was decided to use a basic model and a more advanced model. This was done so the results could be compared easily and identify their strengths and weaknesses. For the basic model, logistic regression was chosen because it is fast to run and train and only required minimal tuning meaning it was fast to implement. For the advanced model, DistilBERT was used because it "reduces model size by 40% … and increases speed by up to 60% while retaining 97% BERT's performance" (Emergent Mind, no date). This was essential as due to the increased complexity meaning it would take a lot longer to train compared to the basic model. DistilBERT was also chosen because it has been pre-trained to understand context from a sentence. This allows it to more accurately predict the emotion as it is bidirectional meaning it can see the words before and after in comparison to simply just counting the number of times a word appears for an emotion with logistic regression.

```
#create multiclass logistic regression model
model_lr = LogisticRegression(
    solver='saga',
    multi_class='multinomial',
    max_iter=1000,
    random_state=42,
    n_jobs=-1
)
```

Figure 10 – Basic model logistic regression

```
#pre-trained model
model_name = "distilbert-base-uncased"
batch_size = 16 #Processes tweets in sets of 16
epochs = 3 # does 3 full runs of the dataset
learning_rate = 5e-5 #how fast it learns
```

Figure 11 – Advanced model DistilBERT

4. Evaluation and Conclusion

In conclusion, the two models both performed well however the DistilBERT produced significantly better results while the logistic regression model ran significantly faster. The DistilBERT classification report shows it had 4% greater accuracy overall. It significantly increased the f1-score of the weaker emotions such as joy (+7%) and surprise (+6%). It also becomes highly reliable with certain emotions such as anger with a 97% chance to get it right. Importantly it increased the macro average by 9% to 90% meaning that even emotions with little data for support still had a good chance of being guessed correctly.

```
Classification Report (for the 6 emotions):
              precision    recall  f1-score   support

   Anger (0)       0.97      0.96      0.97       933
    Fear (1)       0.97      0.92      0.94      1072
     Joy (2)       0.78      0.92      0.85       261
    Love (3)       0.90      0.96      0.93       432
 Sadness (4)       0.95      0.83      0.88       387
Surprise (5)       0.72      0.99      0.83       115

    accuracy                           0.93      3200
   macro avg       0.88      0.93      0.90      3200
weighted avg       0.93      0.93      0.93      3200

Saved  in 'distilbert_emotion_model' directory.
```

Figure 12 - Classification report for DistilBERT (as shown in section 2)

On the other hand, the logistic regression model only took 30 seconds from running to produce the classification report while the DistilBERT took 5 minutes and 40 seconds which is 1130% the amount of time which is massive disadvantage. Despite this, the extra processing time and complexity of the BERT model was worthwhile to improve accuracy of emotion detection. The transformer still struggled with emotions like joy and surprise which could be improved by using a more complex model of BERT however this would come with the potential drawback of increased processing time and required computing power. Another way emotion detection could be improved is by making the support more equal for each emotion by oversampling. Joy only had 261 and surprise had just 115 samples compared to the highest getting 1072 meaning it had nearly 10 times the amount of data to train on. This could significantly improve the performance of the weaker emotions as emotions with high support managed to get as high as 97% accuracy.

Classification Report (for the 6 emotions):

	precision	recall	f1-score	support
Anger (0)	0.92	0.93	0.92	933
Fear (1)	0.88	0.93	0.91	1072
Joy (2)	0.82	0.74	0.78	261
Love (3)	0.90	0.87	0.88	432
Sadness (4)	0.84	0.83	0.84	387
Surprise (5)	0.82	0.73	0.77	115
accuracy			0.89	3200
macro avg	0.87	0.84	0.85	3200
weighted avg	0.89	0.89	0.89	3200

Figure 13 - Classification Report for Logistic Regression (as shown in section 2)

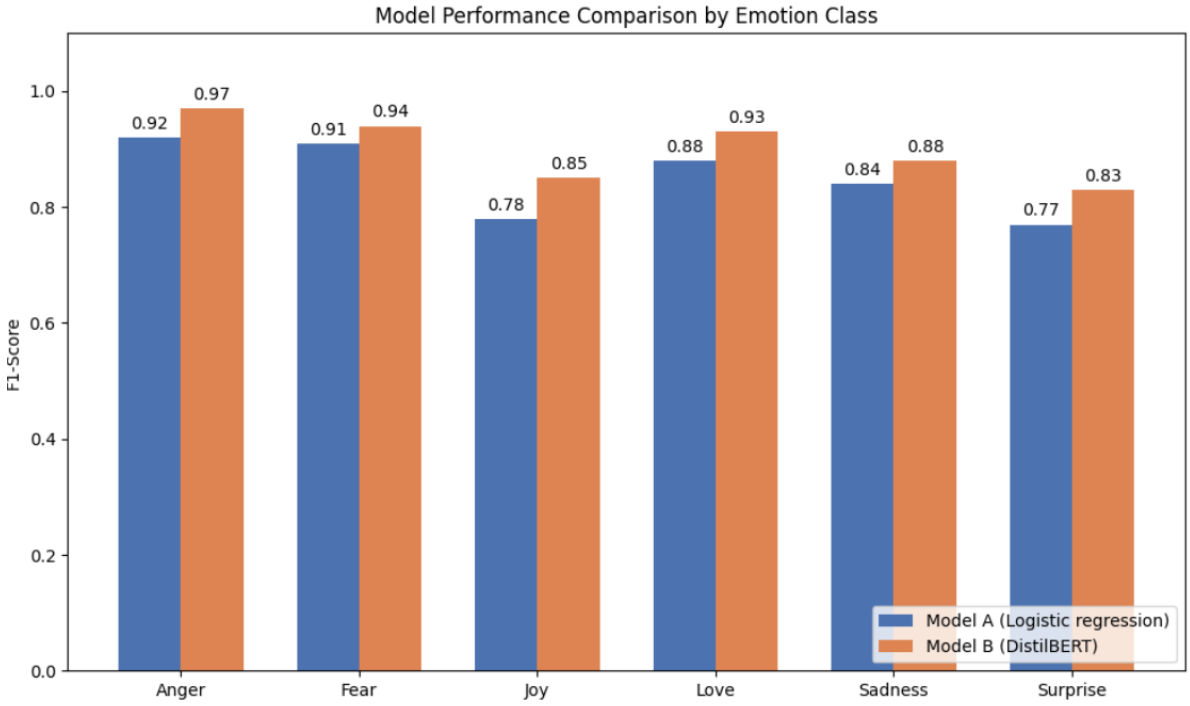


Figure 14 – Graph comparing the model performance for emotions between the two models

5. Innovation

For the innovation, the logistic regression model introduced a feature to show the top 3 words for each emotion. This could be a useful data collecting tool as it allows you to see the words which are influencing each emotion making the model more interpretable as it has clear readable information. This could be used for debugging to see words and if they don't seem relevant to the emotion it could flag that the training has not been successful. Furthermore, the top words for each emotion could be gathered and used as a form of preprocessing in the future. For example, such as the word "pressured" had a high weight for sadness so could be used to help it identify the emotion if the tweet is ambiguous.

```
Top 3 most influential words per emotion

Anger (0):
  - 'troubled' (Weight: 3.0291)
  - 'disturbed' (Weight: 2.9494)
  - 'lethargic' (Weight: 2.9372)

Fear (1):
  - 'innocent' (Weight: 2.6700)
  - 'resolved' (Weight: 2.5936)
  - 'satisfied' (Weight: 2.5467)

Joy (2):
  - 'sympathetic' (Weight: 3.9538)
  - 'nostalgic' (Weight: 3.9404)
  - 'naughty' (Weight: 3.8590)

Love (3):
  - 'greedy' (Weight: 3.5989)
  - 'resentful' (Weight: 3.5943)
  - 'offended' (Weight: 3.4632)

Sadness (4):
  - 'pressured' (Weight: 3.8494)
  - 'shaken' (Weight: 3.6631)
  - 'terrified' (Weight: 3.6284)

Surprise (5):
```

Figure 15 – Shows the top 3 most influential words per emotion and their weighting for the logistic regression model

6. References

Jabreel, M. and Moreno, A. (2019) *A deep learning-based approach for multi-label emotion classification in Tweets*, *MDPI*. Available at: https://www.mdpi.com/2076-3417/9/6/1123 (Accessed: 15 December 2025).

Devlin, J. *et al.* (no date) *Bert: Pre-training of deep bidirectional Transformers for language understanding*, *ACL Anthology*. Available at: https://aclanthology.org/N19-1423/ (Accessed: 15 December 2025).

Poria, S., Cambria, E. and Gelbukh, A. (no date) *Deep convolutional neural network textual features and multiple kernel learning for utterance-level multimodal sentiment analysis*, *ACL Anthology*. Available at: https://aclanthology.org/D15-1303/ (Accessed: 15 December 2025).

Joulin, A. *et al.* (no date) *Bag of tricks for efficient text classification*, *ACL Anthology*. Available at: https://aclanthology.org/E17-2068/ (Accessed: 15 December 2025).

Emergent Mind*: Efficient NLP transformers* (no date) *Trending Papers*. Available at: https://www.emergentmind.com/topics/bert-distilbert (Accessed: 15 December 2025).

7. Annex

Logistic regression model code

```
import re #regex
import pandas as pd #pandas
import random #to shuffle the datasets
from sklearn.feature_extraction.text import CountVectorizer #changes the text into numbers
from sklearn.model_selection import train_test_split #splits the data into training and testing data
from sklearn.linear_model import LogisticRegression #uses multiclass logistic regression
from sklearn.metrics import classification_report, accuracy_score #imports evaluator for logistic
regression
import pickle #to save the model

# get access to the data
df = pd.read_csv("Option1-TrainingDataset.csv")

#Preprocessing
def clean_text(text):
    if not isinstance(text, str): #if not a string replace with empty string
        text = ''

    text = text.lower()
    text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE) # Remove links
    text = re.sub(r'#', '', text) #Remove hashtags but keeps word after it
    text = re.sub(r'[^\w\s]', '', text) # Removes punctuation
```

```
    return text


# add variables for columns to store
texts = df['text'].values
labels = df['label'].values

texts = pd.Series(texts).apply(clean_text).values


#emotion labels to make it clearer in the report than using the numbers
emotion_labels = ['Anger (0)', 'Fear (1)', 'Joy (2)', 'Love (3)', 'Sadness (4)', 'Surprise (5)']

# change the text into numbers
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(texts.astype('U'))

# split training data 80% and testing data 20%
X_train, X_test, y_train, y_test = train_test_split(X, labels, test_size=0.2,
random_state=42,stratify=labels)

#create multiclass logistic regression model
model_lr = LogisticRegression(
    solver='saga',
    max_iter=1000,
    random_state=42,
    n_jobs=-1
)

print("Starting training")
model_lr.fit(X_train, y_train)
print("Training complete.")
print("\n")

#Predict what emotion it should be
y_pred = model_lr.predict(X_test)

#Creates report showing precision, recall, and f1-score for the emotions
print("Classification Report (for the 6 emotions):")
print(classification_report(y_test, y_pred,target_names=emotion_labels))
print("\n")

#get the words from count vectorizer
feature_names = vectorizer.get_feature_names_out()

coefficients = model_lr.coef_

print("\n")
print("Top 3 most influential words per emotion")

# loop through each emotion
```

```
for i, emotion in enumerate(emotion_labels):
    #get the weights and sorts it into order, takes the top 3
    top_3_indices = coefficients[i].argsort()[-3:][::-1]

    #gets the words
    top_3_words = [feature_names[j] for j in top_3_indices]

    #get the weights
    top_3_values = [coefficients[i][j] for j in top_3_indices]

    #prints the top words for each emotion
    print(f"\n{emotion}:")
    for word, value in zip(top_3_words, top_3_values):
        print(f"  - '{word}' (Weight: {value:.4f})")

#save the model and vectorizer
with open('logistic_model.pkl', 'wb') as file:
    pickle.dump(model_lr, file)

with open('count_vectorizer.pkl', 'wb') as file:
    pickle.dump(vectorizer, file)

print("Saved  in 'count_vectorizer' and 'logistic_model' directory")
```

DistilBERT model code

```
import re #regex
import pandas as pd #pandas
import random #to shuffle the datasets
from sklearn.feature_extraction.text import CountVectorizer #changes the text into numbers
from sklearn.model_selection import train_test_split #splits the data into training and testing data
from sklearn.linear_model import LogisticRegression #uses multiclass logistic regression
from sklearn.metrics import classification_report, accuracy_score #imports evaluator for logistic
regression
import pickle #to save the model
import torch # provides tensors (special arrays) and does hard math calculations
from time import time #keeps track of time
from torch.utils.data import Dataset, DataLoader #tells python where the data is and loads it
quickly
from transformers import AutoTokenizer, AutoModelForSequenceClassification # the model
distillbert and a translator so it understands text
from torch.optim import AdamW #optimizer, looks at what can be improved
from tqdm import tqdm #shows a progress bar

#get access to the data
df = pd.read_csv("Option1-TrainingDataset.csv")

#Variables

#emotion labels to make it clearer in the report than using the numbers
```

```
emotion_labels = ['Anger (0)', 'Fear (1)', 'Joy (2)', 'Love (3)', 'Sadness (4)', 'Surprise (5)']
number_of_labels = len(emotion_labels)

#pre-trained model
model_name = "distilbert-base-uncased"
batch_size = 16 #Processes tweets in sets of 16
epochs = 3 # does 3 full runs of the dataset
learning_rate = 5e-5 #how fast it learns

# Tries to use GPU but switches to CPU if not
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
print("\n")

#Preprocessing

def clean_text(text):
   if not isinstance(text, str): #if not a string replace with empty string
      text = ''

   text = text.lower()
   text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE) # Remove links
   text = re.sub(r'#', '', text) #Remove hashtags but keeps word after it
   text = re.sub(r'[^\w\s]', '', text) # Removes punctuation
   return text

#Pytorch emotiondataset class

class EmotionDataset(Dataset):
   #creates the class
   def __init__(self, encodings, labels):
      self.encodings = encodings
      self.labels = labels

   def __getitem__(self, idx):
     #changes the data into tensors for the gpu to understand
      item = {key: torch.tensor(val[idx]) for key, val in self.encodings.items()}
      item['labels'] = torch.tensor(self.labels[idx])
      return item

   #how much stuff is inside
   def __len__(self):
      return len(self.labels)

#runs the clean text function
df['text_cleaned'] = df['text'].astype(str).apply(clean_text)

#assigns variables for the columns
texts = df['text_cleaned'].values
labels = df['label'].values
```

```
# split the data between being used for training and testing
X_train_text, X_test_text, y_train, y_test = train_test_split(
    texts,
    labels,
    test_size=0.2,
    random_state=42,
    stratify=labels
)

# the models translator so it understands inputs
tokenizer = AutoTokenizer.from_pretrained(model_name)
print("Training and testing data split and model translator loaded")

# Converts the text to be understandable by model. Truncates if too long. Adds padding if too
short.
train_encodings = tokenizer(
    list(X_train_text),
    truncation=True,
    padding=True
)
test_encodings = tokenizer(
    list(X_test_text),
    truncation=True,
    padding=True
)

#creates the objects
train_dataset = EmotionDataset(train_encodings, list(y_train))
test_dataset = EmotionDataset(test_encodings, list(y_test))

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=batch_size)
print("Objects created. Ready to train")
print("\n")

# Fine tuning the model

# gets the model and makes it ready for multiclass classification
model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=number_of_labels
)
model.to(device)
# Use AdamW optimizer to change weights to be better
optimizer = AdamW(model.parameters(), lr=learning_rate)

print(f"Fine tuning {model_name} for {epochs} epochs.")

#runs the training process
for epoch in range(epochs):
    model.train() # Set model to training mode
```

```
    start_time = time()

    #starts learning, tqdm does a progress bar to show
    for batch in tqdm(train_loader, desc=f"Epoch {epoch+1}"):
        optimizer.zero_grad() #gets rid of old data

        # Moves stuff to the device
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        labels = batch['labels'].to(device)

        # Makes a guess based on the score for each emotion
        outputs = model(input_ids, attention_mask=attention_mask, labels=labels)
        loss = outputs.loss #calculates how many were wrong in the batch

        #based on the ones that were wrong uses that data to optimise and change the gradients and
weights
        loss.backward()
        optimizer.step()

    end_time = time()
    print(f"Epoch {epoch+1} Complete. Loss: {loss.item():.4f} | Time: {end_time - start_time:.2f}s")
    print("Fine tuning done")
print("\n")

#Evaluation

print("Evaluating")
model.eval()
all_preds = []
all_labels = []

#doesnt need gradients for testing, runs faster
with torch.no_grad():
    #loads testing data
    for batch in tqdm(test_loader, desc="Evaluating"):
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        labels = batch['labels']

        outputs = model(input_ids, attention_mask=attention_mask)
        # Tries to predict which emotion it would be
        predictions = torch.argmax(outputs.logits, dim=-1)

        #changes from tensors to numpy to be used for calculations
        all_preds.extend(predictions.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())

#Creates report showing precision, recall, and f1-score for the emotions
print("Classification Report (for the 6 emotions):")
print(classification_report(all_labels, all_preds, target_names=emotion_labels))
```

```
# Saving the model
model.save_pretrained("distilbert_emotion_model")
tokenizer.save_pretrained("distilbert_emotion_model")
print("Saved  in 'distilbert_emotion_model' directory.")
```